

Procesamiento de Imágenes Médicas

Clase teórico-práctica de 2 horas y 30 minutos

Docente

2026-09-10

Tabla de contenidos

1. Propósito de la clase	2
2. Objetivos de aprendizaje	2
3. Parte I. Teoría: fundamentos del procesamiento de imágenes médicas	3
3.1. 1. ¿Qué es una imagen médica?	3
3.2. 2. Flujo general de solución	3
3.3. 3. Representación digital de la imagen	4
3.4. 4. Preprocesamiento	5
3.5. 5. Segmentación	5
3.6. 6. Validación e interpretación	6
4. Parte II. Práctica guiada en Python	6
4.1. 1. Descripción del caso práctico	6
4.2. 2. Preparación del entorno	6
4.3. 3. Importar librerías	7
4.4. 4. Cargar la imagen médica real	7
4.5. 5. Conversión a escala de grises	7
4.6. 6. Normalización de intensidades	7
4.7. 7. Histograma de intensidades	8
4.8. 8. Mejora de contraste con CLAHE	8
4.9. 9. Filtrado Gaussiano	8
4.10. 10. Comparación visual del preprocesamiento	9
4.11. 11. Segmentación con umbral de Otsu	9
4.12. 12. Limpieza morfológica	10
4.13. 13. Etiquetado de regiones	10
4.14. 14. Extracción de métricas	10
4.15. 15. Selección de la región principal	10
4.16. 16. Visualización de contornos	11
4.17. 17. Comparación completa del pipeline	11
5. Parte III. Actividad de modificación de parámetros	12
5.1. Experimento 1: efecto del suavizado	12
5.2. Experimento 2: efecto del contraste local	13

5.3. Experimento 3: efecto del tamaño mínimo de objetos	13
6. Parte IV. Discusión crítica	13
6.1. 1. Preguntas de discusión	13
6.2. 2. Discusión técnica esperada	14
7. Parte V. Rúbrica de evaluación	14
8. Parte VI. Producto esperado	14
9. Conclusión de la clase	15
10. Anexo: versión compacta del pipeline completo	15
11. Anexo: comando para renderizar a PDF	16

1. Propósito de la clase

Esta clase presenta un flujo introductorio para resolver un problema básico de procesamiento de imágenes médicas. El objetivo no es realizar diagnóstico clínico, sino comprender cómo una imagen médica puede convertirse en un dato cuantitativo mediante operaciones computacionales controladas.

La clase está diseñada para una duración total de **2 horas y 30 minutos**, distribuidas así:

Bloque	Duración	Propósito
Teoría	1 hora	Comprender los fundamentos del procesamiento de imágenes médicas
Práctica guiada	1 hora y 30 minutos	Implementar un pipeline básico en Python usando una imagen médica real
Cierre	Incluido en la práctica	Discutir limitaciones, validación e interpretación

2. Objetivos de aprendizaje

Al finalizar la clase, el estudiante deberá ser capaz de:

1. Explicar qué diferencia una imagen médica de una imagen convencional.
2. Describir el flujo general de procesamiento de imágenes médicas.
3. Cargar y visualizar una imagen médica en Python.
4. Aplicar normalización, mejora de contraste y filtrado.
5. Realizar una segmentación inicial usando umbralización.
6. Limpiar una máscara mediante morfología matemática.
7. Extraer métricas básicas de una región segmentada.
8. Reconocer las limitaciones de una segmentación automática simple.

3. Parte I. Teoría: fundamentos del procesamiento de imágenes médicas

Duración sugerida: **60 minutos**.

3.1. 1. ¿Qué es una imagen médica?

Una imagen médica es una representación visual o volumétrica de estructuras anatómicas, tejidos o procesos fisiológicos. A diferencia de una fotografía convencional, la imagen médica tiene una relación directa con una modalidad de adquisición, una escala física, un contexto clínico o investigativo, y una interpretación especializada.

Algunas modalidades frecuentes son:

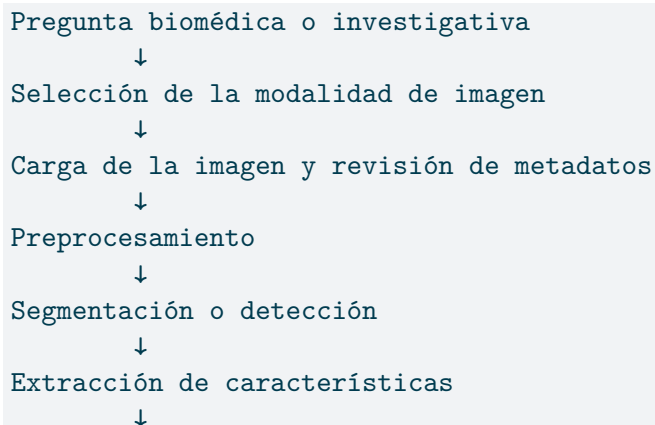
Modalidad	Tipo de información	Ejemplo de uso
Rayos X	Proyección de estructuras densas	Tórax, huesos
Tomografía computarizada	Cortes anatómicos por rayos X	Pulmón, cráneo, abdomen
Resonancia magnética	Contraste de tejidos blandos	Cerebro, músculo, articulaciones
Ecografía	Reflexión de ondas ultrasónicas	Obstetricia, corazón, tejidos blandos
Medicina nuclear	Información metabólica o funcional	PET, SPECT
Histopatología	Imagen microscópica de tejido	Biopsias, análisis celular

En imágenes médicas, los píxeles no son simples colores. Pueden representar atenuación, intensidad de señal, densidad, reflectancia, concentración de trazadores o propiedades asociadas a tejidos.

3.2. 2. Flujo general de solución

Un problema de procesamiento de imágenes médicas debe resolverse de forma ordenada. No se recomienda iniciar directamente aplicando filtros o modelos, porque cada operación debe responder a una pregunta técnica o biomédica concreta.

El flujo general puede expresarse así:



```
Validación
↓
Interpretación y reporte
```

En esta clase se trabajará un flujo simplificado:

```
Imagen médica real
↓
Carga y visualización
↓
Conversión a escala de grises
↓
Normalización
↓
Mejora de contraste
↓
Filtrado
↓
Segmentación
↓
Morfología matemática
↓
Extracción de métricas
```

3.3. 3. Representación digital de la imagen

Una imagen digital puede interpretarse como una matriz numérica.

Para una imagen 2D:

```
I(x, y)
```

Para un volumen 3D:

```
I(x, y, z)
```

Para una imagen dinámica o secuencia temporal:

```
I(x, y, z, t)
```

Conceptos básicos:

Concepto	Descripción
Píxel	Unidad mínima de una imagen 2D
Vóxel	Unidad mínima de una imagen 3D
Intensidad	Valor numérico asociado a cada píxel o voxel
Resolución espacial	Tamaño físico que representa cada píxel
Histograma	Distribución de intensidades de la imagen
Contraste	Diferencia de intensidad entre regiones
Ruido	Variación no deseada de intensidad
Región de interés	Zona relevante para el análisis

Concepto	Descripción
----------	-------------

Antes de procesar una imagen médica, debe analizarse qué representa la intensidad. En una radiografía, una resonancia, una tomografía o una ecografía, los valores no tienen el mismo significado físico.

3.4. 4. Preprocesamiento

El preprocesamiento prepara la imagen para análisis posteriores. No debe aplicarse de manera automática. Cada operación debe tener una justificación.

Operaciones comunes:

Operación	Propósito
Normalización	Llevar intensidades a una escala común
Filtrado Gaussiano	Suavizar ruido de alta frecuencia
Filtro mediana	Reducir ruido impulsivo
Ecualización de histograma	Mejorar contraste global
CLAHE	Mejorar contraste local
Recorte de ROI	Limitar el análisis a una región
Redimensionamiento	Homogeneizar dimensiones

Advertencia técnica:

Mejorar visualmente una imagen no implica necesariamente mejorar su utilidad analítica.
Un filtro puede reducir ruido, pero también puede borrar bordes o detalles relevantes.

3.5. 5. Segmentación

La segmentación consiste en separar una región de interés del fondo o de otras estructuras.

Algunos métodos frecuentes son:

Método	Idea principal	Ventaja	Limitación
Umbral fijo	Separar por intensidad	Simple	Sensible al contraste
Otsu	Umbral automático por histograma	Fácil de aplicar	Falla si las clases no están bien separadas
Bordes	Detectar cambios bruscos de intensidad	Útil para contornos	Sensible al ruido
Crecimiento de regiones	Expandir desde semillas	Intuitivo	Depende de la semilla inicial
Morfología matemática	Limpiar máscaras binarias	Muy útil como postproceso	Depende del tamaño del elemento estructurante
Aprendizaje profundo	Aprende patrones desde datos anotados	Alto potencial	Requiere datos, anotaciones y validación rigurosa

En esta clase se usará una aproximación clásica:

Contraste mejorado → Suavizado → Umbralización → Morfología → Métricas

3.6. 6. Validación e interpretación

Una segmentación no puede considerarse correcta solo porque visualmente parece razonable. En imágenes médicas es necesario comparar los resultados contra una referencia, una anotación experta o un criterio cuantitativo.

Métricas comunes:

Métrica	Uso
Área	Tamaño de la región segmentada
Perímetro	Longitud aproximada del contorno
Intensidad media	Valor promedio dentro de la región
Bounding box	Rectángulo que encierra la región
Dice	Comparación contra máscara de referencia
IoU	Superposición entre máscara automática y referencia
Sensibilidad	Capacidad para detectar regiones positivas
Especificidad	Capacidad para excluir regiones negativas

Idea central:

En imágenes médicas, el algoritmo no termina cuando produce una máscara. Termina cuando esa máscara se interpreta, se valida y se relaciona con una pregunta biomédica concreta.

4. Parte II. Práctica guiada en Python

Duración sugerida: **90 minutos**.

4.1. 1. Descripción del caso práctico

Se trabajará con una imagen médica real en formato PNG, JPG o similar. La imagen debe ser pública, anonimizada y utilizada únicamente con fines educativos.

El objetivo de la práctica es:

Cargar una imagen médica real, mejorar su contraste, aplicar filtrado, realizar una segmentación inicial y extraer métricas básicas de una región segmentada.

4.2. 2. Preparación del entorno

Instalar las librerías necesarias:

```
pip install numpy matplotlib scikit-image scipy
```

También puede usarse un entorno como Google Colab, Jupyter Notebook, Quarto o VSCode.

4.3. 3. Importar librerías

```
import numpy as np
import matplotlib.pyplot as plt

from skimage import io, color, exposure, filters, morphology, measure
from scipy import ndimage as ndi
```

4.4. 4. Cargar la imagen médica real

Antes de ejecutar este bloque, coloque una imagen médica en la misma carpeta del archivo `.qmd`.

Ejemplo de nombre:

```
imagen_medica.png
```

Código:

```
image_path = "imagen_medica.png"

image = io.imread(image_path)

plt.figure(figsize=(6, 6))
plt.imshow(image, cmap="gray")
plt.title("Imagen médica real")
plt.axis("off")
plt.show()
```

4.5. 5. Conversión a escala de grises

Algunas imágenes pueden estar almacenadas en RGB aunque visualmente parezcan estar en escala de grises. Por eso se verifica su dimensionalidad.

```
if image.ndim == 3:
    image_gray = color.rgb2gray(image)
else:
    image_gray = image.astype(float)

plt.figure(figsize=(6, 6))
plt.imshow(image_gray, cmap="gray")
plt.title("Imagen en escala de grises")
plt.axis("off")
plt.show()
```

4.6. 6. Normalización de intensidades

La normalización lleva las intensidades a un rango común entre 0 y 1.

```
image_norm = (image_gray - image_gray.min()) / (image_gray.max() - image_gray.min())

plt.figure(figsize=(6, 6))
```

```
plt.imshow(image_norm, cmap="gray")
plt.title("Imagen normalizada")
plt.axis("off")
plt.show()
```

Preguntas para discusión:

1. ¿Cuál es el rango original de intensidades?
2. ¿Por qué puede ser útil normalizar?
3. ¿La normalización cambia la estructura anatómica de la imagen?

4.7. 7. Histograma de intensidades

El histograma permite observar cómo se distribuyen los valores de intensidad.

```
plt.figure(figsize=(7, 4))
plt.hist(image_norm.ravel(), bins=80)
plt.title("Histograma de intensidades")
plt.xlabel("Intensidad normalizada")
plt.ylabel("Frecuencia")
plt.show()
```

Interpretación esperada:

- Si hay grupos de intensidad bien separados, una segmentación por umbral puede funcionar razonablemente.
- Si las intensidades están muy superpuestas, una umbralización simple puede ser insuficiente.
- En imágenes médicas reales, el histograma rara vez separa de forma perfecta las estructuras anatómicas.

4.8. 8. Mejora de contraste con CLAHE

CLAHE mejora el contraste local. Puede hacer más visibles estructuras que originalmente tienen baja separación de intensidad.

```
image_clahe = exposure.equalize_adapthist(image_norm, clip_limit=0.03)

plt.figure(figsize=(6, 6))
plt.imshow(image_clahe, cmap="gray")
plt.title("Mejora de contraste local con CLAHE")
plt.axis("off")
plt.show()
```

Preguntas para discusión:

1. ¿Qué estructuras se observan mejor después de CLAHE?
2. ¿El contraste mejoró de forma útil o se amplificó el ruido?
3. ¿Qué riesgo existe al modificar demasiado el contraste?

4.9. 9. Filtrado Gaussiano

El filtro Gaussiano suaviza la imagen y puede reducir ruido de alta frecuencia.

```

image_smooth = filters.gaussian(image_clahe, sigma=1.2)

plt.figure(figsize=(6, 6))
plt.imshow(image_smooth, cmap="gray")
plt.title("Imagen suavizada con filtro Gaussiano")
plt.axis("off")
plt.show()

```

Discusión:

Un **sigma** bajo suaviza poco. Un **sigma** alto puede borrar bordes importantes. En procesamiento médico, esta decisión no debe tomarse solo por apariencia visual.

4.10. 10. Comparación visual del preprocesamiento

```

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(image_norm, cmap="gray")
axes[0].set_title("Original normalizada")
axes[0].axis("off")

axes[1].imshow(image_clahe, cmap="gray")
axes[1].set_title("CLAHE")
axes[1].axis("off")

axes[2].imshow(image_smooth, cmap="gray")
axes[2].set_title("CLAHE + Gaussiano")
axes[2].axis("off")

plt.tight_layout()
plt.show()

```

4.11. 11. Segmentación con umbral de Otsu

Otsu calcula automáticamente un umbral a partir del histograma.

```

threshold = filters.threshold_otsu(image_smooth)
mask = image_smooth > threshold

plt.figure(figsize=(6, 6))
plt.imshow(mask, cmap="gray")
plt.title(f"Segmentación inicial con Otsu. Umbral = {threshold:.3f}")
plt.axis("off")
plt.show()

```

Análisis:

Esta máscara no debe asumirse como correcta. Es una primera aproximación. En imágenes médicas reales, un umbral puede separar regiones por intensidad sin que esas regiones correspondan

necesariamente a una estructura anatómica válida.

4.12. 12. Limpieza morfológica

La máscara inicial puede contener puntos aislados, huecos o regiones pequeñas. La morfología matemática permite limpiarla.

```
mask_clean = morphology.remove_small_objects(mask, min_size=500)
mask_clean = morphology.remove_small_holes(mask_clean, area_threshold=500)
mask_clean = morphology.closing(mask_clean, morphology.disk(3))

plt.figure(figsize=(6, 6))
plt.imshow(mask_clean, cmap="gray")
plt.title("Máscara limpiada con morfología matemática")
plt.axis("off")
plt.show()
```

4.13. 13. Etiquetado de regiones

El etiquetado permite identificar regiones conectadas dentro de la máscara.

```
labels = measure.label(mask_clean)

plt.figure(figsize=(6, 6))
plt.imshow(labels, cmap="nipy_spectral")
plt.title("Regiones etiquetadas")
plt.axis("off")
plt.show()
```

4.14. 14. Extracción de métricas

Se calculan propiedades básicas de las regiones segmentadas.

```
regions = measure.regionprops(labels, intensity_image=image_norm)

print(f"Número de regiones detectadas: {len(regions)}")

for i, region in enumerate(regions, start=1):
    print(f"\nRegión {i}")
    print(f"Área en píxeles: {region.area}")
    print(f"Perímetro aproximado: {region.perimeter:.2f}")
    print(f"Intensidad media: {region.mean_intensity:.3f}")
    print(f"Bounding box: {region.bbox}")
```

4.15. 15. Selección de la región principal

Una estrategia simple consiste en seleccionar la región de mayor área.

```
if len(regions) > 0:
    largest_region = max(regions, key=lambda r: r.area)
```

```

print("Región principal detectada")
print(f"Área en píxeles: {largest_region.area}")
print(f"Perímetro aproximado: {largest_region.perimeter:.2f}")
print(f"Intensidad media: {largest_region.mean_intensity:.3f}")
print(f"Bounding box: {largest_region.bbox}")

```

Advertencia:

La región más grande no siempre es la región clínicamente relevante. Esta regla puede servir para una práctica inicial, pero no debe generalizarse sin control.

4.16. 16. Visualización de contornos

Se extraen los contornos de la máscara y se superponen sobre la imagen original.

```

contours = measure.find_contours(mask_clean, level=0.5)

plt.figure(figsize=(7, 7))
plt.imshow(image_norm, cmap="gray")

for contour in contours:
    plt.plot(contour[:, 1], contour[:, 0], linewidth=1.5)

plt.title("Contornos segmentados sobre la imagen original")
plt.axis("off")
plt.show()

```

4.17. 17. Comparación completa del pipeline

```

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

axes[0, 0].imshow(image_norm, cmap="gray")
axes[0, 0].set_title("Original normalizada")
axes[0, 0].axis("off")

axes[0, 1].imshow(image_clahe, cmap="gray")
axes[0, 1].set_title("CLAHE")
axes[0, 1].axis("off")

axes[0, 2].imshow(image_smooth, cmap="gray")
axes[0, 2].set_title("Filtrada")
axes[0, 2].axis("off")

axes[1, 0].imshow(mask, cmap="gray")
axes[1, 0].set_title("Máscara inicial")
axes[1, 0].axis("off")

axes[1, 1].imshow(mask_clean, cmap="gray")

```

```

axes[1, 1].set_title("Máscara limpia")
axes[1, 1].axis("off")

axes[1, 2].imshow(image_norm, cmap="gray")
for contour in contours:
    axes[1, 2].plot(contour[:, 1], contour[:, 0], linewidth=1.5)
axes[1, 2].set_title("Contornos")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

```

5. Parte III. Actividad de modificación de parámetros

Duración sugerida: **20 minutos**.

Los estudiantes deben modificar los siguientes parámetros y observar el efecto sobre la segmentación:

```

# Parámetros para experimentar

sigma_values = [0.5, 1.2, 2.5]
clip_limits = [0.01, 0.03, 0.08]
min_sizes = [100, 500, 1000]

```

5.1. Experimento 1: efecto del suavizado

```

sigma_values = [0.5, 1.2, 2.5]

fig, axes = plt.subplots(1, len(sigma_values), figsize=(15, 5))

for ax, sigma in zip(axes, sigma_values):
    smoothed = filters.gaussian(image_clahe, sigma=sigma)
    ax.imshow(smoothed, cmap="gray")
    ax.set_title(f"Sigma = {sigma}")
    ax.axis("off")

plt.tight_layout()
plt.show()

```

Preguntas:

1. ¿Qué ocurre cuando **sigma** aumenta?
2. ¿Se conserva el borde de las estructuras?
3. ¿Cuál valor parece más razonable para esta imagen?

5.2. Experimento 2: efecto del contraste local

```
clip_limits = [0.01, 0.03, 0.08]

fig, axes = plt.subplots(1, len(clip_limits), figsize=(15, 5))

for ax, clip in zip(axes, clip_limits):
    clahe_test = exposure.equalize_adapthist(image_norm, clip_limit=clip)
    ax.imshow(clahe_test, cmap="gray")
    ax.set_title(f"clip_limit = {clip}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Preguntas:

1. ¿Qué ocurre al aumentar `clip_limit`?
2. ¿Se mejora el contraste de manera útil?
3. ¿Aparecen artefactos o ruido amplificado?

5.3. Experimento 3: efecto del tamaño mínimo de objetos

```
min_sizes = [100, 500, 1000]

fig, axes = plt.subplots(1, len(min_sizes), figsize=(15, 5))

for ax, min_size in zip(axes, min_sizes):
    mask_test = morphology.remove_small_objects(mask, min_size=min_size)
    ax.imshow(mask_test, cmap="gray")
    ax.set_title(f"min_size = {min_size}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Preguntas:

1. ¿Cuántas regiones pequeñas desaparecen?
2. ¿Se elimina ruido o se eliminan estructuras relevantes?
3. ¿Qué criterio debería usarse para escoger este parámetro?

6. Parte IV. Discusión crítica

6.1. 1. Preguntas de discusión

Responder en grupos pequeños:

1. ¿La máscara obtenida representa una estructura anatómica real?
2. ¿Qué errores aparecen en la segmentación?

3. ¿Qué parte del pipeline tiene más impacto sobre el resultado?
4. ¿Qué limitaciones tiene Otsu en una imagen médica real?
5. ¿Qué necesitaríamos para validar la segmentación?
6. ¿En qué casos sería necesario usar aprendizaje profundo?
7. ¿Por qué no debe confundirse procesamiento visual con validez clínica?

6.2. 2. Discusión técnica esperada

La segmentación por umbralización puede ser útil como aproximación inicial, pero presenta limitaciones importantes:

- Depende de la distribución de intensidades.
- Es sensible al ruido.
- Puede fallar cuando diferentes tejidos tienen intensidades similares.
- Puede separar regiones matemáticamente sin significado anatómico.
- No incorpora conocimiento clínico ni espacial avanzado.
- Requiere validación contra una referencia confiable.

Por tanto, una segmentación automática debe entenderse como una hipótesis computacional, no como una verdad clínica.

7. Parte V. Rúbrica de evaluación

Criterio	Puntaje
Comprende el flujo general de procesamiento	20 %
Ejecuta correctamente el código base	20 %
Interpreta histograma, contraste y ruido	20 %
Obtiene una máscara y analiza sus errores	20 %
Discute limitaciones y validación	20 %

8. Parte VI. Producto esperado

Al finalizar la práctica, cada estudiante debe entregar:

1. Imagen médica original cargada.
2. Imagen normalizada.
3. Histograma de intensidades.
4. Imagen con contraste mejorado.
5. Imagen suavizada.
6. Máscara binaria inicial.
7. Máscara limpia.
8. Contornos superpuestos.
9. Métricas básicas de las regiones.
10. Respuesta crítica sobre las limitaciones del método.

9. Conclusión de la clase

El procesamiento de imágenes médicas no es una secuencia mecánica de filtros. Es un proceso técnico que inicia con una pregunta biomédica, continúa con la comprensión de la modalidad de adquisición, aplica transformaciones justificadas, produce mediciones y exige validación.

Una máscara segmentada no es automáticamente un resultado clínico. Es una salida computacional que debe ser evaluada mediante criterios cuantitativos, comparación con referencias y análisis experto.

10. Anexo: versión compacta del pipeline completo

```
import numpy as np
import matplotlib.pyplot as plt

from skimage import io, color, exposure, filters, morphology, measure

# 1. Cargar imagen
image_path = "imagen_medica.png"
image = io.imread(image_path)

# 2. Escala de grises
if image.ndim == 3:
    image_gray = color.rgb2gray(image)
else:
    image_gray = image.astype(float)

# 3. Normalización
image_norm = (image_gray - image_gray.min()) / (image_gray.max() - image_gray.min())

# 4. Mejora de contraste
image_clahe = exposure.equalize_adapthist(image_norm, clip_limit=0.03)

# 5. Filtrado
image_smooth = filters.gaussian(image_clahe, sigma=1.2)

# 6. Segmentación
threshold = filters.threshold_otsu(image_smooth)
mask = image_smooth > threshold

# 7. Limpieza morfológica
mask_clean = morphology.remove_small_objects(mask, min_size=500)
mask_clean = morphology.remove_small_holes(mask_clean, area_threshold=500)
mask_clean = morphology.closing(mask_clean, morphology.disk(3))

# 8. Etiquetado y métricas
labels = measure.label(mask_clean)
regions = measure.regionprops(labels, intensity_image=image_norm)
```

```

print(f"Número de regiones detectadas: {len(regions)}")

if len(regions) > 0:
    largest_region = max(regions, key=lambda r: r.area)

    print("Región principal")
    print(f"Área: {largest_region.area}")
    print(f"Perímetro: {largest_region.perimeter:.2f}")
    print(f"Intensidad media: {largest_region.mean_intensity:.3f}")

# 9. Contornos
contours = measure.find_contours(mask_clean, level=0.5)

# 10. Visualización final
plt.figure(figsize=(7, 7))
plt.imshow(image_norm, cmap="gray")

for contour in contours:
    plt.plot(contour[:, 1], contour[:, 0], linewidth=1.5)

plt.title("Segmentación sobre imagen original")
plt.axis("off")
plt.show()

```

11. Anexo: comando para renderizar a PDF

Desde la terminal, en la carpeta donde está el archivo .qmd:

```
quarto render clase_procesamiento_imagenes_medicas.qmd --to pdf
```

Si desea ejecutar los bloques de Python al renderizar, cambie en el YAML:

```

execute:
  eval: true

```

Sin embargo, para generar una guía PDF estable, se recomienda mantener:

```

execute:
  eval: false

```

De esta forma el documento se compila como material de clase aunque la imagen médica aún no esté disponible en la carpeta.